CASE STUDY
SYNTAX-DIRECTED DEFINITIONS AND SEMANTIC

ANALYSIS

Course Code: CSA1407
Course Title: COMPILER DESIGN

Submitted by
V.Santhosh kumar
Register Number: 192411015

Department of Computer Science and Engineering

Case Study 1

1.1 Introduction

Semantic analysis is the phase of a compiler that follows syntax analysis and is responsible for verifying that the structure produced by the parser is meaningful, and for building an internal representation that later phases (such as intermediate code generation) can use. For arithmetic expressions, this internal representation most commonly takes the form of a syntax tree — either a parse tree annotated with attributes, or a more compact abstract syntax tree (AST) in which only the operators and operands appear as nodes. This case study builds the syntax tree for the expression a + b * c - d, defines a suitable syntax-directed definition (SDD) for the underlying grammar, evaluates the attributes for the given input, and explains how the resulting tree enforces the correct order of operations.

1.1 Grammar Used

To respect the required precedence (multiplication binds tighter than addition and subtraction) and to enforce left associativity for all three operators, the expression grammar is written as a layered grammar with one non-terminal per precedence level:

E → E + T | E - T |

T

T → T * F | T / F |

F

F → (E) | id

Here E (expression) handles addition and subtraction, T (term) handles multiplication and division, and F (factor) handles parenthesised sub-expressions and identifiers/constants.

Because E and T are left-recursive (E → E + T, T → T * F), operators at the same precedence

level are automatically grouped from left to right, which gives left associativity. Because

multiplication/division is generated one level below addition/subtraction, a * or / is always
applied before a + or - unless parentheses override it, which gives the required precedence.

(i)

Construction of the Syntax Tree for a + b * c - d

Parsing a + b * c - d with the grammar above proceeds by first reducing the expression to its
outermost operator. Because - and + are both generated by the left-recursive rule E → E ± T,
and the expression is scanned left to right, the outermost (root) operator is the last + or -
encountered, which is the subtraction. Its left operand is the sub-expression (a + b * c)
reduced through E, and its right operand is the term d. Within (a + b * c), the outermost
operator is the addition; its left operand is the factor a and its right operand is the term (b * c),
inside which multiplication combines b and c. This produces the following abstract syntax
tree, drawn with the root at the top:

-

/ \

+

d

/ \

a

*

/ \

b

c

Reading the tree bottom-up: the leaf nodes b and c combine first under *, the result combines
with a under +, and finally that result combines with d under -. This bottom-up reading is
exactly the order in which the expression must be evaluated, which is the defining property of
an (abstract) syntax tree — internal nodes are operators, leaves are operands, and the shape of
the tree (not any parenthesisation in the source text) fixes the order of evaluation.

(ii)

Syntax-Directed Definition (SDD) for the Grammar

An SDD attaches semantic rules to each production of the grammar. For expression

evaluation and syntax-tree construction, each non-terminal carries one synthesized attribute:

val for the computed numeric value, and node (or ptr) for a pointer to the newly constructed

tree node. The SDD is S-attributed, since every attribute is synthesized (computed only from

the attributes of the children), which makes it directly usable with a bottom-up (LR) parser.

Production

Semantic Rule

E → E1 + T

E.val = E1.val + T.val E.node = mknode('+', E1.node, T.node)

E → E1 - T

E.val = E1.val - T.val E.node = mknode('-', E1.node, T.node)

E → T

E.val = T.val E.node = T.node

Production

Semantic Rule

T → T1 * F

T.val = T1.val * F.val T.node = mknode('*', T1.node, F.node)

T → T1 / F

T.val = T1.val / F.val T.node = mknode('/', T1.node, F.node)

T → F

T.val = F.val T.node = F.node

F → (E)

F.val = E.val F.node = E.node

F → id

F.val = id.entry.value F.node = mkleaf(id, id.entry)

Here mknode(op, left, right) allocates an internal tree node labelled with operator op and links it to its two children, while mkleaf(id, entry) allocates a leaf node referring to the identifier's symbol-table entry. Because every rule only reads attributes of the symbols on the right-hand side of its own production and writes an attribute of the left-hand non-terminal, the definition qualifies as S-attributed and can be evaluated naturally in a single bottom-up pass, attaching actions to the end of each production.

(iii) Attribute Evaluation for a + b * c - d

Assume, for the purpose of illustrating numeric evaluation, that a = 4, b = 2, c = 3 and d = 5 (any values may be substituted; the tree structure and rule sequence remain identical). The attributes are evaluated bottom-up, in the order the productions are reduced:

Step

Production Reduced

Rule Applied

Result

1

F → id (a)

F.val = 4

F.val = 4

2
T → F
T.val = F.val
T.val = 4
3
F → id (b)
F.val = 2
F.val = 2
4
F → id (c)
F.val = 3
F.val = 3
5
T → T1 * F
T.val = 2 * 3
T.val = 6
6
E → T
E.val = 4 (from step 2)
E.val = 4
7
E → E1 + T
E.val = 4 + 6
E.val = 10
8
F → id (d)
F.val = 5
F.val = 5
9
T → F
T.val = F.val
T.val = 5
10
E → E1 - T
E.val = 10 - 5
E.val = 5
The final synthesized attribute of the start symbol, E.val = 5, is the value of the complete

expression. Note that steps 1–5 compute the multiplication b * c before it is combined with a
(step 7), and the subtraction of d (step 10) is performed last — exactly as required by

precedence and left-to-right, left-associative evaluation.

(iv)

How the Syntax Tree Represents the Correct Order of Operations

● Precedence: multiplication is nested one level deeper in the tree (it is a child of the + node rather than a sibling of it), so any tree-walking evaluator that processes children before parents will always compute b * c before combining it with a. Depth in the tree, not left-to-right position in the source text, is what fixes precedence.
● Associativity: because - is the root and its left child is the entire (a + b * c) subtree while its right child is the single leaf d, the tree shows that the addition and multiplication are grouped together first and the subtraction is applied last to the whole group — i.e., (a + b*c) - d rather than a + (b*c - d). This left-to-right grouping of same-precedence operators is the graphical expression of left associativity.
● Evaluation order: a post-order traversal of the tree (children first, then the node itself) produces exactly the evaluation sequence shown in the attribute-evaluation table, so the tree can be handed directly to a code generator or an interpreter without any further analysis of the original text.
● Independence from surface syntax: once built, the tree no longer contains parentheses or any notion of grammar rules — it is a pure representation of "what must be computed before what", which is precisely the internal representation a compiler needs for later phases such as intermediate-code generation or optimisation.

1.2 Parse Tree Versus Abstract Syntax Tree

It is useful to distinguish the full parse tree (also called a concrete syntax tree) that a strict reading of the grammar would generate from the compact abstract syntax tree (AST) shown in Section 1.2(i). A full parse tree for a + b * c - d using the layered grammar E/T/F would include a node for every application of every production — separate E, T and F nodes even where a rule such as T → F or E → T does nothing but pass a value through unchanged. An AST omits these "chain" nodes and keeps only nodes that carry real operators or operands, because the mknode / mkleaf actions in the SDD are written to skip them. The table below contrasts the two representations for the given expression.

Aspect

Parse Tree

Abstract Syntax Tree (used above)	
Node for every grammar rule applied	
Yes, including E → T and T → F chain rules	
No — chain productions are collapsed	
Number of nodes for a + b * c - d	
13 nodes (E,T,F at every level)	
7 nodes (-, +, *, a, b, c, d)	

Aspect
Parse Tree
Abstract Syntax Tree (used above)
Purpose
Mirrors the derivation exactly;
used to verify grammar
correctness
Used by later compiler phases (code
generation, optimisation)
Built by
The parser itself, implicitly
The SDD's mknode/mkleaf actions,
explicitly
Because the case study asks specifically for "an internal representation using syntax trees" for
later semantic processing, the AST form is the appropriate deliverable, and it is the one constructed and evaluated in Sections 1.2(i) and 1.2(iii).

1.3 A Second Worked Example

To confirm that the grammar and SDD generalise correctly, consider a second expression built from the same grammar: p - q * r + s. Applying the same reasoning as before — the outermost operator is the last + or - scanned left to right at the E level, so the tree is built as
follows:

Step 1: q * r is a T, formed first (highest precedence).
Step 2: p - (q*r) is formed next, since - appears before + and
both are at the E level,
so left associativity forces the leftmost E-level operator
to be applied first.
Step 3: the result is then combined with s using +.

+
/ \
-
s
/ \
p
*

/ \

q

r

This confirms the general rule embodied by the grammar: at the E level, - and + are resolved

strictly left to right (left associativity), while any * or / nested underneath is always evaluated

first (precedence) — exactly the same two properties demonstrated for a + b * c - d, now shown to hold for a differently-ordered mix of the same operators.

1.4 Common Implementation Pitfalls

● Using a single flat rule such as E → E + E | E - E | E * E | id (no separate T and F levels) is ambiguous — it does not encode precedence at all, and a parser generator would report shift-reduce conflicts or silently pick an arbitrary, possibly incorrect,

order.

● Writing the addition/multiplication rules as right-recursive (E → T + E) instead of left-recursive (E → E + T) still parses the same language but yields right associativity, which gives the wrong grouping for expressions such as a - b - c (it would compute a - (b
- c) instead of the correct (a - b) - c).

● Forgetting to build a distinct tree node for each operator application, and instead reusing the same node object across recursive calls, is a common bug that silently corrupts previously built parts of the tree — mknode must always allocate a fresh node.
● Omitting the F → (E) production, or forgetting to make F.val simply equal to the inner E.val, would prevent parenthesised sub-expressions from ever overriding the default precedence, defeating one of the main purposes of having explicit grouping symbols in the language.

1.5 Relevance in Production Compilers

The technique illustrated here — a precedence-layered grammar combined with an S-attributed SDD that incrementally builds an AST — is essentially the same strategy used
inside real compiler front ends such as GCC and Clang/LLVM, which construct an AST during or immediately after parsing and then hand that tree to subsequent phases (type checking, intermediate-representation generation, and optimisation passes) that no longer need to know anything about the original grammar or token stream. The correctness of every
later phase therefore depends on the syntax tree faithfully capturing precedence and associativity exactly as demonstrated in this case study.

1.6 Key Takeaways

● Precedence is enforced by grammar layering (E over T over F), not by any special-casing in the semantic rules themselves.
● Left associativity is enforced by left recursion (E → E + T, T → T * F); switching to right recursion would silently change the associativity.
● The SDD in Section 1.2(ii) is S-attributed, so it evaluates naturally alongside bottom-up parsing with no lookahead or backpatching required.
● The resulting syntax tree is the compiler's internal representation of the expression and is what later phases (code generation, optimisation) actually consume — its shape, not the original source text, is what encodes the correct order of operations.

Case Study 2

S-Attributed Definitions and Bottom-Up Evaluation with a Parser Stack

2.1 Why This Grammar Suits Bottom-Up (S-Attributed) Evaluation

In an S-attributed SDD every attribute is synthesized: its value is computed only from the attribute values of the symbols on the right-hand side of the production that produces it. This property matches the way an LR (bottom-up, shift-reduce) parser works, because at the moment a production is reduced, all of its right-hand-side symbols are already on the parser stack with their attributes already computed. The rule for S can therefore fire the moment E n is reduced, printing or returning the final value with no need to look ahead or to pass information downward into a subtree — which is exactly what "synthesized-only" means.

(i)

S-Attributed SDD

Production	Semantic Rule
S → E n	print(E.val)
E → E1 + T	E.val = E1.val + T.val
E → E1 - T	E.val = E1.val - T.val
E → T	E.val = T.val
T → T1 * F	T.val = T1.val * F.val
T → T1 / F	T.val = T1.val / F.val
T → F	T.val = F.val
F → (E)	F.val = E.val
F → digit	F.val = digit.lexval

(ii)
Translator Code Fragment

A stack-based translator implements this SDD by keeping a parallel semantic stack, val[ ],
alongside the parser's state stack. Each time the parser reduces by a production, the
corresponding action below is executed, using top to refer to the current top of the semantic
stack before the reduction:

case E -> E1 + T : val[ntop] = val[top-2] + val[top]
case E -> E1 - T : val[ntop] = val[top-2] - val[top]
case E -> T
: val[ntop] = val[top]
case T -> T1 * F : val[ntop] = val[top-2] * val[top]
case T -> T1 / F : val[ntop] = val[top-2] / val[top]
case T -> F
: val[ntop] = val[top]
case F -> (E) : val[ntop] = val[top-1]
case F -> digit : val[ntop] = lexval case
S -> E n
: print(val[top])
After each reduction the matched right-hand-side entries are popped and replaced by the
single computed value for the left-hand non-terminal, so the semantic stack always mirrors
the parser's symbol stack one-for-one. This is the standard technique for combining
bottom-up parsing with on-the-fly intermediate-code / value generation without building an
explicit tree in memory.
(iii) Step-by-Step Evaluation Using the Parser Stack for 8 + 2 * 4 n

Step
Stack (symbols)
Input
Remaining
Action
Semantic Value /
Computation
1
$
8+2*4n
shift 8

—
2
$ 8
+2*4n
reduce F → digit
F.val = 8
3
$ F
+2*4n
reduce T → F
T.val = 8
4
$ T
+2*4n
reduce E → T
E.val = 8
5
$ E
+2*4n
shift +
—
6
$ E +
2*4n
shift 2
—
7
$ E + 2
*4n
reduce F → digit
F.val = 2
8
$ E + F
*4n
reduce T → F
T.val = 2
9
$ E + T

*4n
shift *
—
10
$ E + T *
4n
shift 4
—
11
$ E + T * 4
n
reduce F → digit
F.val = 4
12
$ E + T * F
n
reduce T → T1*F
T.val = 2*4 = 8
13
$ E + T
n
reduce E → E1+T
E.val = 8+8 = 16
14
$ E
n
shift n
—

Step
Stack (symbols)
Input
Remaining
Action
Semantic Value /
Computation
15
$ E n
$
reduce S → E n
print(16)
The key moment is step 12: because * has higher precedence than + in the grammar hierarchy
(T is nested inside E), the parser reduces T → T1 * F — computing 2 * 4 = 8 — before it ever
reduces E → E1 + T in step 13. The shift-reduce parser therefore never has the opportunity to
add 8 + 2 first; the grammar's structure forces multiplication to complete before the pending
addition is resolved, giving the mathematically correct result 8 + (2 * 4) = 16.
(iv)
Annotated Syntax Tree for 8 + 2 * 4
E.val = 16
+
/
\
E.val=8
T.val=8
|
*
T.val=8
/
\
|
T.val=2
F.val=4 F.val=8

|

|

|

F.val=2

'4'

'8'

|

'2'

Every node in the tree is labelled with its synthesized val attribute. The value at the root, 16,
is obtained only after both children's values (8 and 8, the second being the product 2*4) are
available — which mirrors exactly the bottom-up order of reduction traced in the stack table
above.

(v)

How Bottom-Up S-Attributed Evaluation Ensures Correct Precedence and Associativity

● Precedence is encoded structurally: because T (the level for * and /) is defined strictly beneath E (the level for + and -) in the grammar, an LR parser is forced to shift and reduce every T before the surrounding E-level reduction can occur, so multiplicative sub-expressions are always fully reduced to a single value before an additive rule combines them.

● Associativity is encoded by left recursion: rules such as E → E1 + T and T → T1 * F place the "already built" left context inside the non-terminal itself, so repeated operators of the same precedence (e.g. a + b + c) are reduced left to right, producing

((a+b)+c) rather than (a+(b+c)).

● S-attributed rules match the parser's natural evaluation order: because a bottom-up parser only ever has completed sub-trees on its stack at the moment of a reduction, synthesized-only attributes can always be computed immediately, with no need for lookahead into not-yet-parsed input or backpatching — this is why S-attributed SDDs are the natural fit for LR parsing and why the value computed on reduction is guaranteed to already respect precedence and associativity.

2.5 Generating Intermediate Code (Three-Address Code) for the Same Input

Beyond computing a single final numeric value, the same S-attributed framework is normally extended so that each non-terminal's attribute is not the value itself but the name of a temporary variable holding that value, which lets the translator emit three-address code as a by-product of parsing. The rule for T → T1 * F, for instance, becomes T.place = newtemp(); emit(T.place = T1.place '*' F.place), and similarly for the other arithmetic productions. Applying this version of the SDD to 8 + 2 * 4 produces the following three-address code, generated in the same order as the reductions in the stack table of Section 2(iii):

Quadruple #
Generated Instruction
Corresponding Reduction
1
t1 = 2 * 4
T → T1 * F (step 12)
2
t2 = 8 + t1
E → E1 + T (step 13)

Only two temporaries are needed because the grammar's S-attributed structure guarantees that every sub-expression is reduced to a single named value before it is used by an enclosing operator — t1 holds the product 2*4 = 8, and t2 = 8 + t1 = 16 holds the final result, matching the value 16 obtained by direct evaluation in Section 2(iii).

2.6 Comparison with a Top-Down Approach for the Same Grammar

It is instructive to note why bottom-up, S-attributed evaluation was chosen here rather than a top-down approach. The grammar E → E + T | E - T | T is left-recursive, so a naive top-down

(recursive-descent) parser cannot use it directly; it would need first to be rewritten into a right-recursive form with inherited attributes (the technique used later in Case Study 5) before
top-down translation becomes possible. Bottom-up LR parsing, by contrast, handles left recursion natively and evaluates S-attributed rules with no grammar transformation at all, which is precisely why this case study specifies a stack-based bottom-up parser for the original, left-recursive grammar.

2.7 Common Implementation Pitfalls

● Forgetting to pop the correct number of semantic-stack entries on each reduction (the right-hand side of T → T1 * F has three symbols, so three entries must be popped and replaced by one) silently misaligns the semantic stack with the parser's symbol stack for every subsequent reduction.
● Evaluating * and + with the same table of parser actions without respecting the grammar's precedence levels (i.e. flattening E and T into one non-terminal) would allow the parser to reduce E → E + T before the following * F has been consumed, producing the incorrect result (8+2)*4 = 40 instead of 16.
● Using array indices that assume a fixed offset from the stack top without accounting for productions of different lengths (e.g. F → (E) has three symbols but only the middle one contributes a value) is a frequent off-by-one source of bugs in hand-written translators.

2.8 Key Takeaways

● An S-attributed SDD attaches a single synthesized attribute per non-terminal and computes it entirely from the right-hand-side symbols already on the parser stack at reduction time.
● The stack trace in Section 2(iii) shows precedence being enforced procedurally: T → T1
* F is always reduced before the pending E → E1 + T, so multiplication is always folded in before addition.
● The same reduction sequence that computes the final value can simultaneously emit three-address code, giving the compiler both a checked result and translator output from a single bottom-up pass.
● Bottom-up (LR) parsing is the natural home for S-attributed definitions precisely because it handles left-recursive grammars — such as the one given in this case study — without any rewriting.

2.9 Conclusion

This case study has shown, end to end, how a compiler front end can evaluate an arithmetic
expression and simultaneously produce translator output using nothing more than a bottom-up parser and a set of synthesized semantic rules. The S-attributed SDD of Section
2(i) was implemented as a semantic stack in Section 2(ii), traced instruction-by-instruction
against the input 8 + 2 * 4 n in Section 2(iii), rendered as an annotated tree in Section 2(iv),
and finally used

to emit compact three-address code in Section 2.5. Every stage confirms the same underlying
fact: because the grammar places T strictly inside E, and the parser always reduces inner constituents before outer ones, correct precedence and left associativity fall out automatically
from the parsing order, with no special-case logic required in the semantic actions themselves.

Case Study 3
Classifying Attribute-Rule Sets as S-Attributed, L-Attributed, or Circular

Q3. A compiler developer is designing a semantic analyzer using syntax-directed definitions (SDD). The analyzer must determine whether given attribute rules follow valid evaluation strategies such as S-attributed or L-attributed definitions, and whether they can be evaluated without conflicts.
Production: A → B C D. Each non-terminal (A, B, C, D) has a synthesized attribute s and an inherited attribute i.
Set (i): A.s = B.i + C.i
Set (ii): A.s = B.i + C.s ; D.i = A.i + B.s
Set

(iii):
A.s
=
B.s
+
D.s
Set (iv): A.s = D.i ;
B.i = A.s + D.s ;
C.i = B.s
;
D.i =
B.i
+ C.i
Tasks: (i) S-attributed check. (ii) L-attributed check. (iii) Evaluation order / circular-dependency check. (iv) Justify via dependency relationships. (v) Identify circular dependencies
and explain why they block evaluation.

3.1 Definitions Used for Classification

● S-attributed: every rule computes a synthesized attribute of the left-hand non-terminal purely from attributes of the symbols on the right-hand side (no rule may define, or depend on, an inherited attribute).
● L-attributed: for the production A → X1 X2 … Xn, each inherited attribute of Xj may depend only on: (a) attributes of X1, …, Xj-1 (symbols to its left), and (b) inherited attributes of A itself. Synthesized attributes may depend on anything to their left as usual. Every S-attributed definition is automatically L-attributed, but not vice versa.
● Circular dependency: occurs when the dependency graph of a production contains a cycle, i.e. an attribute's value is (directly or indirectly) required in order to compute itself. A circular SDD cannot be evaluated at all, under any order.
Set (i): A.s = B.i + C.i

Check
Result
Reason
S-attributed?
No
A.s depends on B.i and C.i, which are inherited attributes of

the children — an S-attributed rule may use only synthesized attributes of the RHS symbols.

L-attributed?
No
L-attributedness restricts how INHERITED attributes of B and C may be defined; but here B.i and C.i are used, not defined, and no rule anywhere in this set defines them. Since B.i and C.i are never computed, the definition is incomplete/undefined for those attributes, so it cannot be certified as L-attributed either.

Check
Result
Reason
Circular?
No cycle, but incomplete
There is no cycle among the stated rules, but B.i and C.i have no defining rule at all, so A.s cannot actually be evaluated — the SDD as given is simply underspecified rather than circular.
Justification: the single rule shown only tells us how A.s is computed once B.i and C.i are known; it says nothing about how B.i and C.i themselves get their values. A complete SDD must supply defining rules for every attribute that is used. As it stands, this rule set is not evaluable — not because of a cycle, but because two of the attributes it depends on are never defined anywhere in the given rules.
Set (ii): A.s = B.i + C.s ; D.i = A.i + B.s

Check
Result
Reason
S-attributed?
No
A.s uses B.i, an inherited attribute of a child — not allowed under S-attributed rules, which permit only synthesized attributes of RHS symbols.
L-attributed?
Partially — needs B.i and

B.s to be
defined
elsewher
e
D.i = A.i + B.s is fine for L-attributedness: D is the 3rd symbol
in A → B C D, and it depends on B.s (to its left) and A.i
(inherited attribute of the parent) — both permitted sources.
But A.s = B.i + C.s again uses B.i, and B.i is never defined by
any rule in this set, so the set is incomplete in the same way as
Set (i).
Circular?
No cycle
detected
Among the two rules given there is no attribute that depends,
even indirectly, on itself. The problem is a missing definition
for B.i, not a cycle.
Justification: taken purely as a dependency check, D.i correctly draws only from B.s (a left
sibling's synthesized attribute) and A.i (the parent's inherited attribute), which is a textbook
example of a legal L-attributed inherited rule. However, the set as a whole is still missing a
defining rule for B.i, which A.s needs; until that is supplied the set cannot be evaluated,
though it is structurally consistent with the L-attributed discipline.
Set (iii): A.s = B.s + D.s

Check
Result
Reason

S-attributed?

Yes

A.s depends only on B.s and D.s, both synthesized attributes of RHS symbols (B and D) of the production A → B C D. No inherited attribute is read or written anywhere in this rule.

L-attributed?

Yes

Every S-attributed SDD is automatically L-attributed, since the (empty) set of inherited-attribute restrictions is trivially satisfied.

Circular?

No

The dependency graph is simply B.s → A.s and D.s → A.s: a small directed acyclic graph (DAG) with no path returning to

Check

Result

Reason

any node already visited.

Justification: this is the cleanest of the four sets. It can be evaluated with a single bottom-up (post-order) pass — compute B.s and D.s first (from their own subtrees), then apply the rule to obtain A.s — which is exactly how an LR parser would evaluate it on reduction of A → B C D.

Set (iv): A.s = D.i ; B.i = A.s + D.s ; C.i = B.s ; D.i = B.i + C.i

This set must be checked carefully because several rules reference each other's results. Build the dependency edges first (an edge X → Y means "Y needs X's value"):

- A.s = D.i ⇒ D.i → A.s
- B.i = A.s + D.s ⇒ A.s → B.i and D.s → B.i
- C.i = B.s ⇒ B.s → C.i
- D.i = B.i + C.i ⇒ B.i → D.i and C.i → D.i

Now trace the chain starting from D.i: D.i → A.s → B.i → D.i. This is a directed cycle: D.i is

required to compute A.s, A.s is required to compute B.i, and B.i is required to compute D.i —
bringing us back to the attribute we started from. In other words, D.i indirectly depends on
itself.

Check
Result
Reason
S-attributed?
No
B.i, C.i and D.i are inherited attributes defined by these rules,
and A.s itself depends on D.i, an inherited attribute — this
immediately violates the synthesized-only restriction.

L-attributed?

No

Even ignoring the cycle, C.i = B.s is fine (B is to the left of C), but B.i = A.s + D.s depends on D.s, and D is to the RIGHT of B in the production A → B C D — an inherited attribute of B may not depend on a symbol positioned after it. This alone breaks the L-attributed left-to-right ordering rule.

Circular?

Yes

D.i → A.s → B.i → D.i forms a directed cycle in the attribute dependency graph.

Justification and explanation of the cycle: an attribute dependency graph must be a DAG for any evaluation order to exist, because evaluating an attribute requires first evaluating everything it depends on. Here, to compute D.i we must first know B.i and C.i; to compute B.i we must first know A.s and D.s; and to compute A.s we must first know D.i — the very attribute we set out to compute. No sequence of evaluation steps can break this loop: whichever of D.i,

A.s, or B.i is attempted first, the rule that defines it demands a value that has not yet been produced. This is precisely why Set (iv) is circular and therefore cannot be evaluated by any
parsing strategy (top-down or bottom-up) without first rewriting the grammar or the rules to
remove the cycle.

3.2 Summary Table

Rule Set	S-Attributed	L-Attributed	Evaluable (Acyclic)
(i)	No	No (incomplete)	No definer for B.i, C.i
(ii)	No	Structurally L-attributed, but incomplete	No definer for B.i
(iii)	Yes	Yes	Yes — DAG, evaluable bottom-up
(iv)	No	No (right-dependency + cycle)	No — circular dependency

3.3 General Algorithm for Detecting a Circular SDD

In practice, a compiler-compiler (such as Yacc or an attribute-grammar tool) checks every
production automatically rather than relying on manual tracing. The standard procedure is: (1)

build a dependency graph for each production, with one node per attribute occurrence and a directed edge from attribute X to attribute Y whenever the rule computing Y reads the value of X; (2) attempt a topological sort of this graph; (3) if the topological sort succeeds, that order is a valid attribute-evaluation order; (4) if the algorithm ever reaches a point where every remaining node still has at least one incoming edge from another remaining node, the graph contains a cycle and the SDD is declared circular. Applying this procedure to Set (iv) is exactly what the trace in Section 3.1 did manually: attempting to schedule D.i, A.s and B.i in any order always leaves at least one of the three waiting on another that is waiting on it, so no topological order exists.

3.4 Repairing a Circular Definition

A circular SDD cannot be fixed by choosing a cleverer evaluation order — the only remedies are to change the rules or the grammar. Two general strategies are used in practice:

● Re-derive the offending rule so that it no longer needs a value that is not yet available — for example, if B.i genuinely only needs information already known before D is parsed, redefine B.i using only A.i and attributes of symbols to B's left, removing the dependency on D.s entirely.

● Restructure the grammar itself (e.g. reorder the symbols on the right-hand side, or introduce an auxiliary non-terminal) so that the natural left-to-right, parent-to-child flow of an L-attributed definition becomes sufficient, avoiding the need for a symbol to depend on information from a sibling positioned after it.

Because Set (iii) already demonstrates a rule set that is both S-attributed and free of any inherited attribute, it represents the simplest and safest style to aim for whenever a redesign is possible: restrict semantic rules to synthesized attributes computed purely from children wherever the required computation allows it.

3.5 Key Takeaways

● S-attributed definitions are a strict subset of L-attributed definitions, which are themselves a strict subset of all SDDs; Set (iii) is S-attributed (and hence L-attributed and acyclic), while Sets (i) and (ii) are incomplete and Set (iv) is genuinely circular.

● An L-attributed inherited attribute may only look left (to earlier siblings) or up (to the parent's inherited attribute) — never right, to a sibling appearing later in the same production; Set (iv)'s B.i = A.s + D.s violates this directly.

● A dependency graph with a cycle has no valid topological order, so no evaluator — top-down, bottom-up, or otherwise — can compute the attributes in Set (iv) without the rules or the grammar first being changed.

● Detecting circularity is done algorithmically in real tools via topological sorting of the per-production dependency graph, exactly as illustrated by hand in Section 3.3.

3.6 Conclusion

Classifying a rule set as S-attributed, L-attributed, or circular is one of the first correctness checks a compiler designer must perform before choosing a parsing and evaluation strategy, because the classification directly determines which parsers can even evaluate the definition at all. This case study worked through four representative situations for the production A → B C D: an incomplete definition with undefined inherited attributes (Set (i)), a partially correct but still incomplete L-attributed sketch (Set (ii)), a fully valid S-attributed definition ready for bottom-up evaluation (Set (iii)), and a genuinely circular definition that no evaluator could ever process (Set (iv)). Recognising these patterns by tracing dependency edges, as demonstrated

throughout Section 3, is the same discipline that attribute-grammar tools automate when validating a language's semantic rules before a compiler is generated from them.

Case Study 4

Propagating Inherited Type Attributes During Bottom-Up Parsing of Declarations

4.1 The Nature of the Problem

Unlike Case Studies 1 and 2, this problem is not about computing a numeric value — it is about type propagation: the single type keyword (real) must be communicated to every identifier in the comma-separated list that follows it, so that the compiler can insert the correct type into each identifier's symbol-table entry. Because the type is declared once but
must reach several identifiers scattered across the list, this is a classic inherited-attribute problem: information flows downward and sideways from T into L, rather than bubbling upward as a single synthesized value.

(i)

Syntax-Directed Definition

Production	Semantic Rule
D → T L	L.in = T.type
T → int	T.type = integer
T → real	T.type = real
L → L1 , id	L1.in = L.in ; addtype(id.entry, L.in)
L → id	addtype(id.entry, L.in)

T.type is synthesized (computed from the keyword actually matched). L.in is inherited: it is
set once, from T.type, when D → T L is applied, and is then copied down to every L on the
left of a comma via L1.in = L.in, so that ultimately every id in the list receives the same type
through repeated calls to addtype.

(ii)

Inherited and Synthesized Attributes and Their Roles

Attribute	Kind	Role
T.type	Synthesized	Records which base type (int or real) was matched by the keyword; computed purely from the terminal symbol, with no dependency on context.
L.in	Inherited	Carries the type decided by T down into the identifier list L, so every id reduction can look up the correct type; its value flows from parent/left-context into the subtree rather than being built up from children.
id.entry	(reference)	A pointer into the symbol table for the specific identifier being declared; not computed by the grammar but used as the target that addtype writes into.

The role of L.in captures the essential idea of inherited attributes: a single piece of context-dependent information (the declared type) must be made visible to several nodes that do not themselves compute it — L.in simply passes that information along, unchanged, to each id it reaches.

(iii) & (v) Bottom-Up Evaluation and Stack Implementation for real p, q, r

A genuine LR parser reduces strictly bottom-up and left to right, so it reduces T → real (giving T.type) before it has seen any of p, q, r, and it reduces L → id for p before the later commas and identifiers are even shifted. This means that when L → id (for p) is reduced, the final value of L.in (which is only formally assigned when D → T L eventually fires) is not literally available yet in a naive reading of the SDD above. In a practical bottom-up implementation, this is resolved by not waiting for the D → T L reduction to logically "deliver" L.in; instead, the parser keeps T.type in a accessible position on the semantic stack (immediately below the symbols of L) the moment T is reduced, and each L → id / L → L1,

id action reads it directly from that fixed stack offset rather than through a formally passed-down inherited value. This stack-offset technique is the standard way inherited attributes of this restricted, left-to-right shape are realised in an LR parser without rewriting
the grammar into a purely S-attributed form.

Step
Stack (symbols / values)
Remaining Input
Action
Attribute Evaluation
1
$
real p , q , r
shift real
—
2
$ real
p , q , r
reduce T → real
T.type = real (kept on stack)
3
$ T
p , q , r
shift p
—
4
$ T p
, q , r
reduce L → id
addtype(p, T.type=real)

5
$ T L
, q , r
shift ,
—

Step
Stack (symbols / values)
Remaining Input
Action
Attribute Evaluation
6
$ T L ,
q , r
shift q
—
7
$ T L , q
, r
reduce L → L1 , id
L1.in = T.type ;
addtype(q, real)
8
$ T L
, r
shift ,
—
9
$ T L ,
r
shift r
—
10
$ T L , r
$

reduce L → L1 , id
L1.in = T.type ;
addtype(r, real)
11
$ T L
$
reduce D → T L
L.in confirmed = T.type
= real
By the end of step 11 all three identifiers p, q and r have had addtype(id, real) executed against them, so the symbol table records the correct type for every declared variable, and the reduction D → T L simply confirms that the propagated type used throughout was indeed T.type.
(iv)
Annotated Parse Tree
D
/
\
T.type=real
L.in=real
|
/
|
\
'real'
L.in=real ',' id(r)
/
|
\
addtype(r,real) L.in=real ',' id(q)
|
addtype(q,real) id(p)
addtype(p,real)
Each L node in the tree is annotated with the same in = real value, inherited from the single T node at the top, and each id leaf triggers one addtype call that writes real into that identifier's symbol-table entry. The tree makes visually explicit what the stack trace showed

procedurally: one synthesized fact (the keyword's type) is broadcast, unchanged, to every identifier in the list.

4.2 Why This Is Harder Than a Purely Synthesized Problem

Sections 4(iii) and 4(v) already noted that L.in is, strictly speaking, an inherited attribute that

a pure bottom-up parser cannot compute in the textbook top-down sense, because inherited attributes are normally expected to be pushed down from a parent node before its children are visited, whereas an LR parser visits (reduces) children before their parent. The reason the propagation nevertheless works correctly for this specific grammar is that T (the source of the type) always appears strictly to the left of L in the input, so by the time the parser begins

reducing any part of L, T has already been fully reduced and its value is permanently available at a fixed, known position on the parser stack. This is a special case sometimes called a copy rule situated at the left edge of a production — it is one of the few shapes of inherited attribute that bottom-up parsers can support without full attribute-grammar machinery, precisely because no value from later in the input is ever required.

4.3 Alternative Implementation: A Global Variable

A simpler, commonly used implementation avoids semantic-stack bookkeeping altogether by keeping a single global variable, say currentType, that is set when T is reduced and simply read (not passed as a formal attribute) whenever L → id or L → L1, id fires:

T -> int
{ currentType = integer
} T -> real { currentType = real }
L -> id
{ addtype(id.entry, currentType)
} L -> L1 , id
{ addtype(id.entry, currentType)
}

This produces identical results for the declaration real p, q, r, and is easier to implement in a hand-written parser, but it is less general: it silently breaks if declarations can be nested or if two declarations are ever processed concurrently, because currentType is shared global state rather than an attribute scoped to a particular parse subtree. The formal SDD with L.in given in Section 4(i) is preferred in a robust compiler precisely because it scopes the type information correctly to each declaration.

4.4 Common Implementation Pitfalls

● Confusing which symbol the type keyword modifies when a language allows multiple declarations on adjacent lines — without correctly scoping L.in (or resetting currentType) per declaration statement D, a type from one line can leak into the next.

● Calling addtype before the type is known (e.g. if the grammar or parser table mistakenly allowed L to be reduced before T), which would insert identifiers into the symbol table with an undefined or default type.

● Failing to check for duplicate identifiers within the same list (e.g. real p, p, r) — addtype should also verify that an identifier has not already been declared in the

current scope and raise a semantic error if so.

4.5 Relevance to Real Symbol-Table Management

This pattern — one type specifier propagated across a comma-separated identifier list — is exactly how declarations are processed in the semantic-analysis phase of languages such as C, C++ and Pascal. The symbol table entries created by addtype in this case study are subsequently consulted throughout the rest of compilation: for type checking every expression that uses p, q or r, for computing storage sizes and stack offsets, and for detecting type errors such as assigning a string to a variable declared real. The correctness of all of that later work depends entirely on the type propagation illustrated here being applied consistently to every identifier in the declaration.

4.6 Key Takeaways

- L.in is an inherited attribute that must be defined once (from T.type) and copied unchanged down the left-recursive chain of L productions so every identifier receives it.
- This particular inherited attribute is evaluable during bottom-up parsing only because its source, T, always appears strictly to the left of L in the input, letting the parser keep T.type accessible on the stack throughout the reduction of L.
- The stack trace in Section 4(iii)/(v) shows addtype firing once per identifier (p, q and r), each time using the same real type recorded when T was reduced.
- The same mechanism generalises directly to real compilers' symbol-table construction for variable declarations in languages such as C and Pascal.

4.7 Conclusion

Propagating a declared type to every identifier in a list is a small but representative example of the broader class of inherited-attribute problems that arise throughout semantic analysis — the same left-to-right, "broadcast a fact down and across the tree" pattern reappears when propagating scope information, propagating array-dimension declarations, or propagating access modifiers in object-oriented languages. This case study showed the full pipeline for the specific declaration real p, q, r: the formal SDD with an inherited L.in (Section 4.1), the practical bottom-up realisation of that inherited value via a stack-offset technique (Section 4.2), a worked parser-stack trace that fires addtype exactly once per identifier (Section 4.3/4.5), and the resulting annotated parse tree (Section 4.4) — together demonstrating that

inherited attributes, while conceptually associated with top-down evaluation, can be implemented correctly in a bottom-up parser whenever their source of information always
appears to the left of where it is needed.

Case Study 5
Top-Down Translation of Arithmetic Expressions Using L-Attributed Definitions

Q5. A compiler developer is designing a semantic analyzer that performs top-down translation of arithmetic expressions using Syntax-Directed Definitions (SDD). The analyzer must evaluate arithmetic expressions and construct an annotated parse tree during
parsing.
Grammar: S → E n
|
E → E + T | T
|
T → T * F | F
|
F → (E) | digit
Input
expression:
3
*
4
+
6
n
Tasks: (i) Construct an SDD using L-attributed definitions. (ii) Show how inherited and synthesized attributes are propagated top-down. (iii) Perform the top-down translation and
compute the final value. (iv) Construct the annotated parse tree with attribute values at each
node.

5.1 Eliminating Left Recursion for Top-Down Parsing

The grammar as given (E → E + T | T, T → T * F | F) is left-recursive, and a top-down (predictive, recursive-descent) parser cannot process left recursion directly — it would recurse infinitely trying to expand E before consuming any input. The standard transformation replaces left recursion with right recursion using an auxiliary non-terminal per
level (traditionally written E' and T'), while an inherited attribute carries the "value accumulated so far" forward through the tail of the expression:

S
→ E n
E
→ T E'
E' → + T { ... } E'
|
ε
T
→ F T'
T' → * F { ... } T'
|
ε
F
→ (E)
|
digi
t
This rewritten grammar generates exactly the same language and preserves precedence (T nested inside E) and left associativity (achieved now through the inherited-attribute chain in
E' and T' rather than through left recursion), while being suitable for LL(1) top-down parsing.
(i)
L-Attributed SDD

Production
Semantic Rule
S → E n
print(E.val)
E → T E'
E'.inh = T.val ; E.val = E'.val
E' → + T E'1
E'1.inh = E'.inh + T.val ; E'.val = E'1.val
E' → ε
E'.val = E'.inh

T → F T'

T'.inh = F.val ; T.val = T'.val

Production	Semantic Rule
T' → * F T'1	T'1.inh = T'.inh * F.val ; T'.val = T'1.val
T' → ε	T'.val = T'.inh
F → (E)	F.val = E.val
F → digit	F.val = digit.lexval

E'.inh and T'.inh are inherited attributes (the running total flows into each E' / T' from its left sibling or parent); E'.val, T'.val, E.val, T.val and F.val are synthesized (the final total flows back up once the ε-production is reached). Because each inherited attribute here depends only on attributes of symbols to its left in the same production (or on the parent's inherited attribute), and each synthesized attribute depends only on the children, the definition satisfies the L-attributed discipline exactly — which is what allows it to be evaluated during a single left-to-right, top-down parse.

(ii)

Propagation of Inherited and Synthesized Attributes Top-Down

● Inherited attributes travel downward and rightward: when E → T E' is applied, T is parsed and its synthesized value T.val is immediately handed down as E'.inh — the accumulator for "everything added so far" — before E' itself is expanded further.

● Each recursive expansion of E' (via + T E'1) updates the accumulator by adding the next term's value, then passes the updated accumulator down to the next E'1, moving strictly left to right through the + operators — this left-to-right threading is what reproduces left associativity without left recursion.

● The chain terminates at E' → ε, where there is nothing left to add; the ε-production simply copies the inherited accumulator into the synthesized result, E'.val = E'.inh, which then flows back up through every level of the recursion as each E'1.val is

copied up to become its E'.val, until it reaches the outermost E.val.

● Exactly the same inherited/synthesized pairing happens one level down for T and T' with multiplication, so a * chain is fully resolved into a single T.val before that value is ever handed to E' as one term of the addition.

(iii) Top-Down Translation of 3 * 4 + 6

Predictive parsing expands the input left to right; the attribute computation below is shown
interleaved with the expansions that consume each token.

Step
Expansion / Token Consumed
Attribute Action
Value
1
S → E n
start
—
2
E → T E'
T parses '3 * 4'
T.val = 12 (see step 3)
3
T → F T'
F → digit(3): F.val = 3
T'.inh = 3
4
T' → * F T'1
consume '*', F → digit(4): F.val = 4
T'1.inh = 3 * 4 = 12
5
T'1 → ε
no more '*'
T'1.val = 12 ⇒ T'.val = 12 ⇒ T.val = 12
6
E'.inh = T.val
hand T's result to E'
E'.inh = 12
7
E' → + T E'1
consume '+', T → digit(6): T.val = 6
E'1.inh = 12 + 6 = 18

8
E'1 → ε
no more '+'
E'1.val = 18 ⇒ E'.val
= 18
9
E.val = E'.val
propagate upward
E.val = 18
10
S → E n : print(E.val)
consume 'n'
Output = 18
The multiplication 3 * 4 is fully collapsed into T.val = 12 (steps 3–5) before that value is ever
combined with anything at the E level; only afterwards, in step 7, is it added to 6, giving the
correct final value 3 * 4 + 6 = 18, matching standard operator precedence.
(iv)
Annotated Parse Tree
S
|
E.val = 18
'n'
/
\
T.val = 12
E'.inh=12, E'.val=18
/

\
/
|

\
F.val=3
T'.inh=3,T'.val=12 '+'
T.val=6
E'1.inh=18,E'1.val=18

Every E' and T' node carries a pair of attribute values — inh (received from the left) and val
(returned to the right/upward) — showing precisely how the running total 3, then 12, then 18
is threaded through the tree from left to right and finally surfaces as the synthesized value
E.val = 18 at the root, which S then prints as the result of translating the input 3 * 4 + 6.

5.4 Why Eliminating Left Recursion Still Preserves Left Associativity

It may seem surprising that a right-recursive grammar (E' → + T E'1) can still produce left-associative results, since right recursion is normally associated with right associativity.

The reason is that the grammar's shape controls only the shape of the parse tree, not the order in which values are combined — the SDD is what fixes evaluation order. Here, the inherited attribute E'.inh always represents "everything computed so far, from the left", and each new term is folded into that accumulator with E'1.inh = E'.inh + T.val, i.e. the previously accumulated (left) value is always the first operand and the newly parsed (right) term is always added to it. This left-to-right folding is mathematically identical to left associativity, even though the underlying parse tree grows down and to the right. It is a standard technique precisely because it lets a top-down parser, which cannot handle left recursion, still faithfully reproduce left-associative semantics.

5.5 A Second Worked Example, Including Parentheses

To confirm the SDD handles grouping correctly, consider the input (3 + 4) * 5 n. Parsing proceeds as S → E n, E → T E', and T → F T', where F must first match the parenthesised sub-expression:

- F → (E): parsing the inner E for 3 + 4 recursively invokes the same SDD — T.val = 3, E'.inh = 3, then + 4 gives E'1.inh = 3 + 4 = 7, so the inner E.val = 7 and therefore F.val = 7.
- Back at the outer level, T'.inh = F.val = 7, then * 5 gives T'1.inh = 7 * 5 = 35, so T.val = 35.
- Since there is no following + at the outer E level, E'.val = E'.inh = T.val = 35, so E.val = 35, and S prints 35.

This matches direct evaluation of (3 + 4) * 5 = 7 * 5 = 35, confirming that F → (E) correctly allows a parenthesised group to be evaluated as an independent, self-contained expression (via a fresh, nested application of the same E → T E' rule) before its value is used by the enclosing T' as an ordinary factor — exactly the mechanism that lets parentheses override default precedence.

5.6 Common Implementation Pitfalls

● Forgetting to pass E'.inh downward before recursing into the next E'1 (e.g. writing E'1.inh = T.val instead of E'.inh + T.val) discards everything computed so far and silently produces only the value of the last term instead of the full running sum.
● Implementing the ε-production incorrectly by leaving E'.val undefined instead of copying the inherited value (E'.val = E'.inh) breaks the chain that carries the final accumulated result back up to E.val — a very common bug in first attempts at this transformation.
● Mixing up which attribute is inherited and which is synthesized when translating the informal rules into a recursive-descent parser in code — a function implementing E' must accept inh as a parameter (since it is inherited) and return val as its result (since it is synthesized); reversing this convention is a frequent source of confusion for students new to L-attributed translation.
● Attempting to apply this SDD with a bottom-up (LR) parser without modification: L-attributed definitions that rely on inherited attributes flowing strictly left-to-right are designed for top-down evaluation; some, but not all, L-attributed SDDs can also be evaluated bottom-up with extra care, but the translation scheme above is presented specifically for, and is most naturally executed by, a top-down predictive parser.

5.7 Key Takeaways

● Left recursion must be eliminated before top-down parsing; the standard transformation introduces E' and T' with an inherited accumulator attribute that reconstructs left-associative semantics despite the grammar now being right-recursive.
● The SDD in Section 5(i) is L-attributed: every inherited attribute depends only on attributes to its left or on the parent's inherited attribute, which is exactly what a left-to-right, top-down parse can supply as it goes.
● The worked translation of 3 * 4 + 6 (Section 5.3) confirms that * is folded into T.val = 12 before it reaches the E level, so the final result 18 correctly respects precedence.
● The same scheme extends immediately to parenthesised sub-expressions, as shown in Section 5.5, because F → (E) simply invokes a fresh, self-contained instance of the same E → T E' rule.